

Detailed Design

Zeeshan Ali Rana

Design Methodology

- Design decisions are periodically revisited and revised
- Refactoring
 - to simplify complicated solutions or to optimize the design

Design Principles

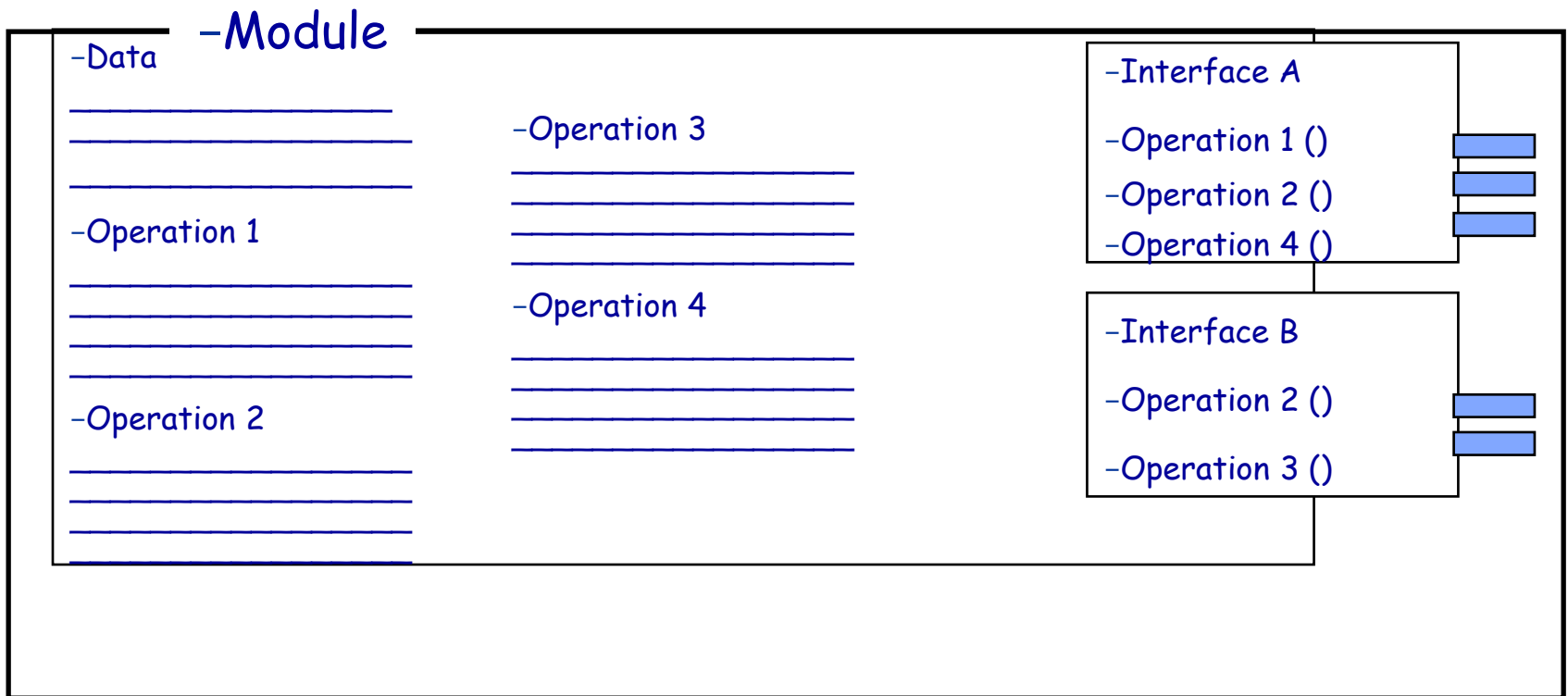
- **Design principles** are guidelines for decomposing a system's required functionality and behavior into modules
- The principles identify the criteria
 - for decomposing a system
 - deciding what information to provide (and what to conceal) in the resulting modules
- Six dominant principles (general):
 - Interfaces
 - Information hiding
 - Incremental development
 - Abstraction
 - Generality
 - Modularity

Interfaces

- An **interface** defines what services the software unit provides to the rest of the system, and how other units can access those services
 - For example, the interface to an object is the collection of the object's public operations and the operations' **signatures**, which specify each operation's name, parameters, and possible return values
- An interface must also define what the unit requires, in terms of services or assumptions, for it to work correctly
- A software unit's interface describes what the unit requires of its environment, as well as what it provides to its environment

Interfaces (Contd.)

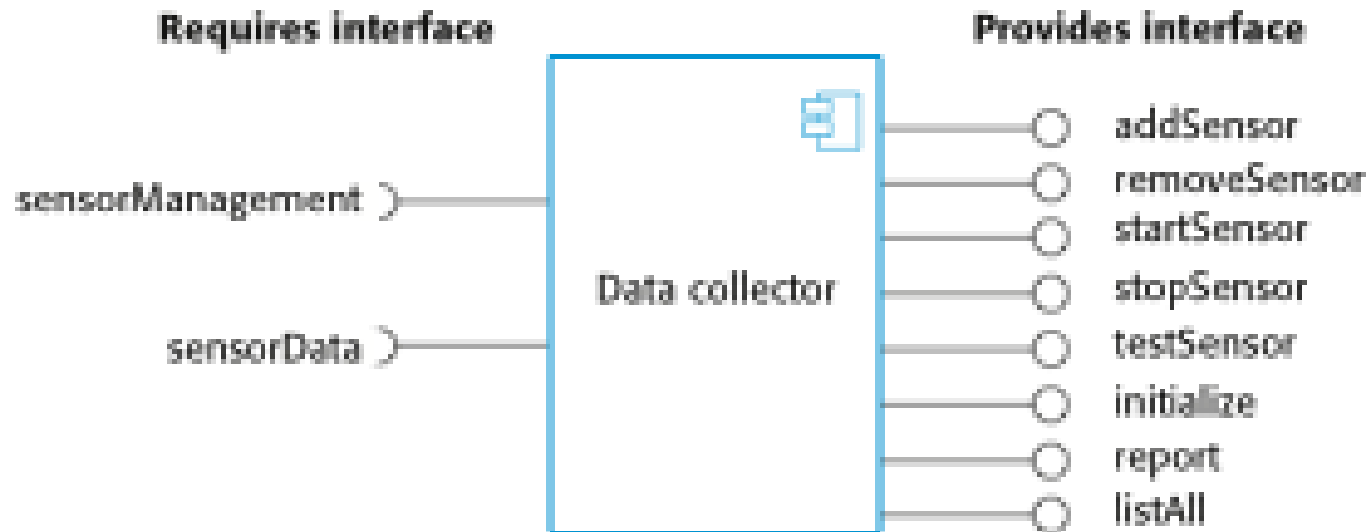
- A software unit may have several interfaces that make different demands on its environment or that offer different levels of service



Interfaces (Contd.)

- The **specification** of a software unit's interface describes the externally visible properties of the software unit
- An interface specification should communicate to other system developers everything that they need to know to use our software unit correctly
 - Purpose (document the functionality that will help to select what fits the need)
 - Preconditions (assumptions- libraries, input)
 - Protocols (e.g calling module may need to be authorized before accessing resource)
 - Postconditions (visible effects- return values, outputs)
 - Quality attributes (e.g a client may want to know whether internal data structures have been optimized for data insertion or retrieval (optimization of one usually slows down other))

A Component with Interfaces



Information Hiding

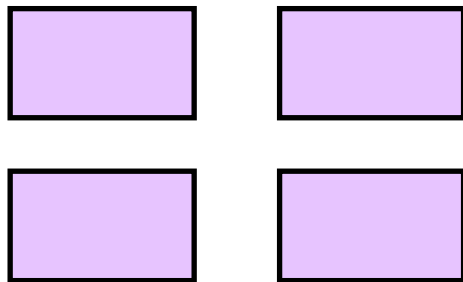
- **Information hiding** is distinguished by its guidance for decomposing a system:
 - Each software unit encapsulates a separate design decision that could be changed in the future
 - Then the interfaces and interface specifications are used to describe each software unit in terms of its externally visible properties (to help its reuse)
- Using this principle, modules may exhibit different kinds of cohesion
 - A module that hides a data representation may be informationally cohesive
 - A module that hides an algorithm may be functionally cohesive
- Advantages of information hiding is that the resulting software units are loosely coupled, and programmer know only what they need to know

Modularity

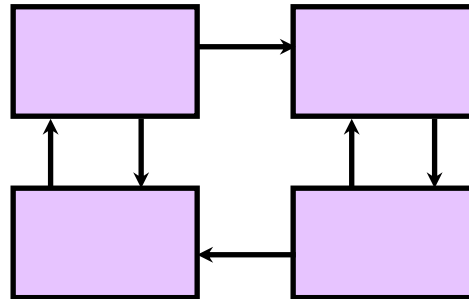
- **Modularity** is the principle of keeping the unrelated aspects of a system separate from each other,
 - each aspect can be studied in isolation (also called separation of concerns)
- If the principle is applied well, each resulting module will have a single purpose and will be relatively independent of the others
 - each module will be easy to understand and develop
 - easier to locate faults
 - because there are fewer suspect modules per fault
 - Easier to change the system
 - because a change to one module affects relatively few other modules
- To determine how well a design separates concerns, we use two concepts that measure module independence: coupling and cohesion

Modularity: Coupling

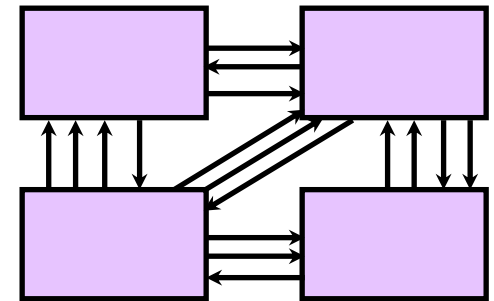
- Two modules are **tightly coupled** when they depend a great deal on each other
- **Loosely coupled** modules have some dependence, but their interconnections are weak
- **Uncoupled** modules have no interconnections at all; they are completely unrelated



-Uncoupled -
-no dependencies



-Loosely coupled -
-some dependencies



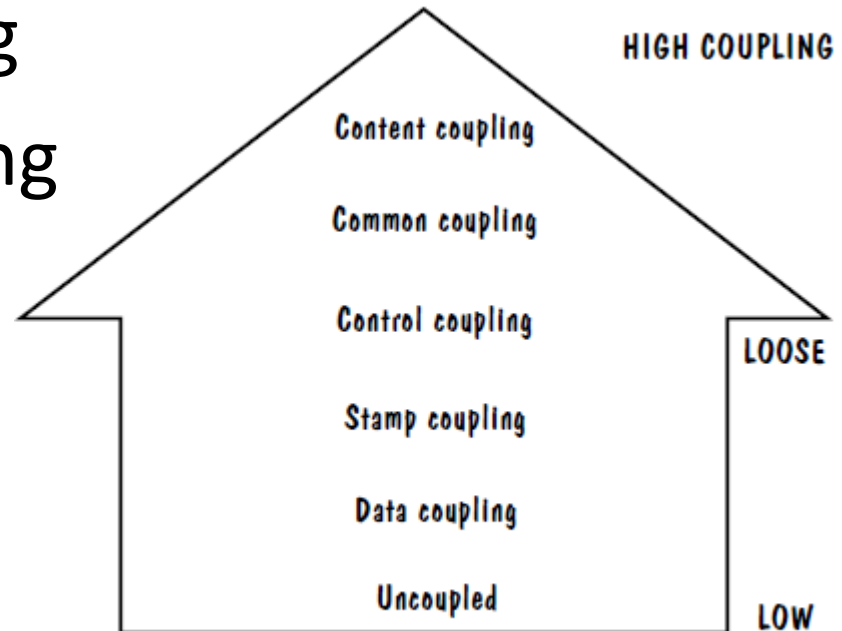
-Tightly coupled -
-many dependencies

Modularity: Coupling

- There are many ways that modules can depend on each other:
 - The references made from one module to another
 - The amount of data passed from one module to another
 - The amount of control that one module has over the other
- Coupling can be measured along a spectrum of dependence

Modularity: Coupling: Types of Coupling

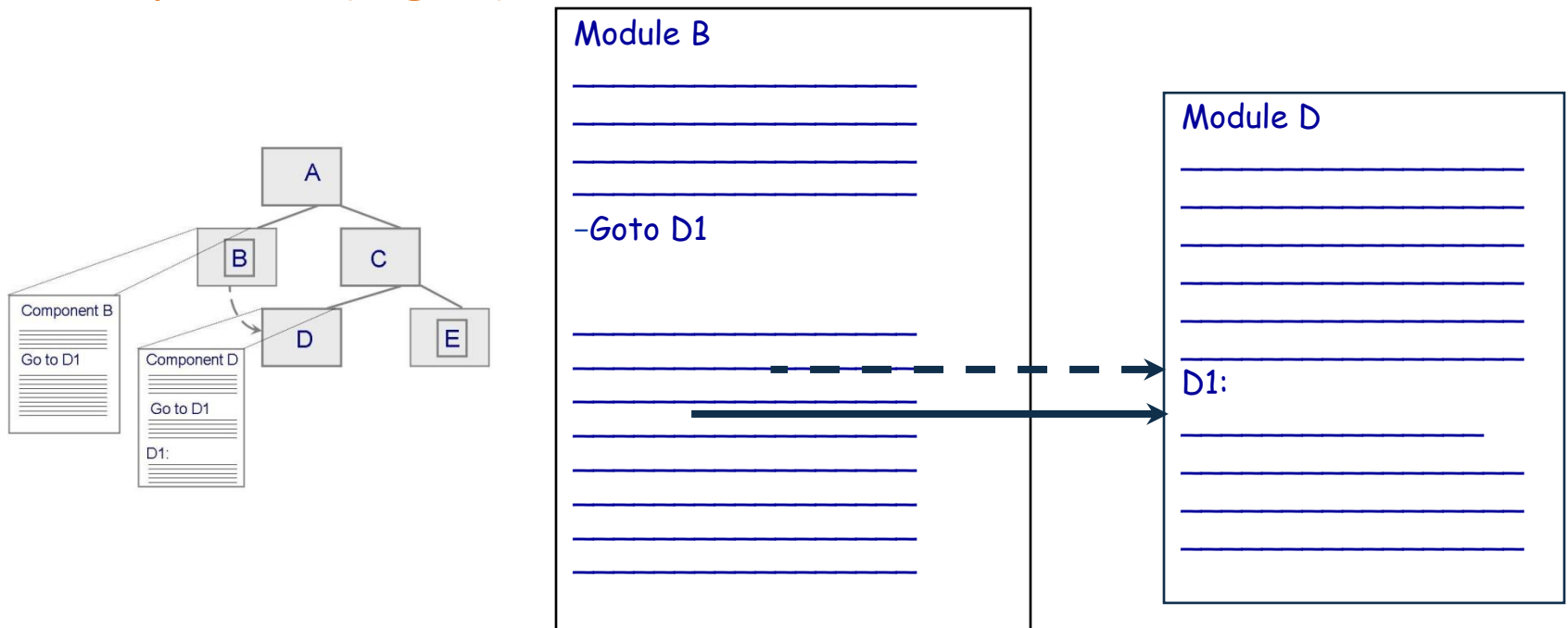
- Content coupling
- Common coupling
- Control coupling
- Stamp coupling
- Data coupling



High coupling is not desired

Modularity: Coupling: Content Coupling

- One component modifies an internal data item of another component, a component (e.g. B) branches into another component (e.g. D)

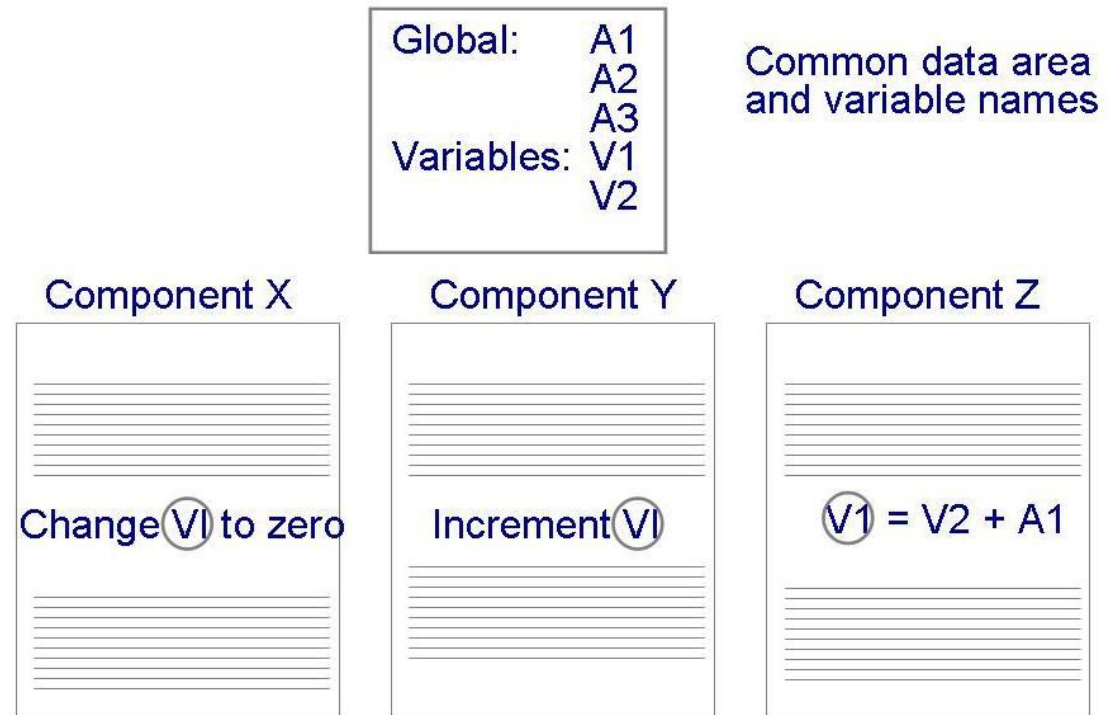


Modularity: Coupling: Common Coupling

- Dependency due to common data.

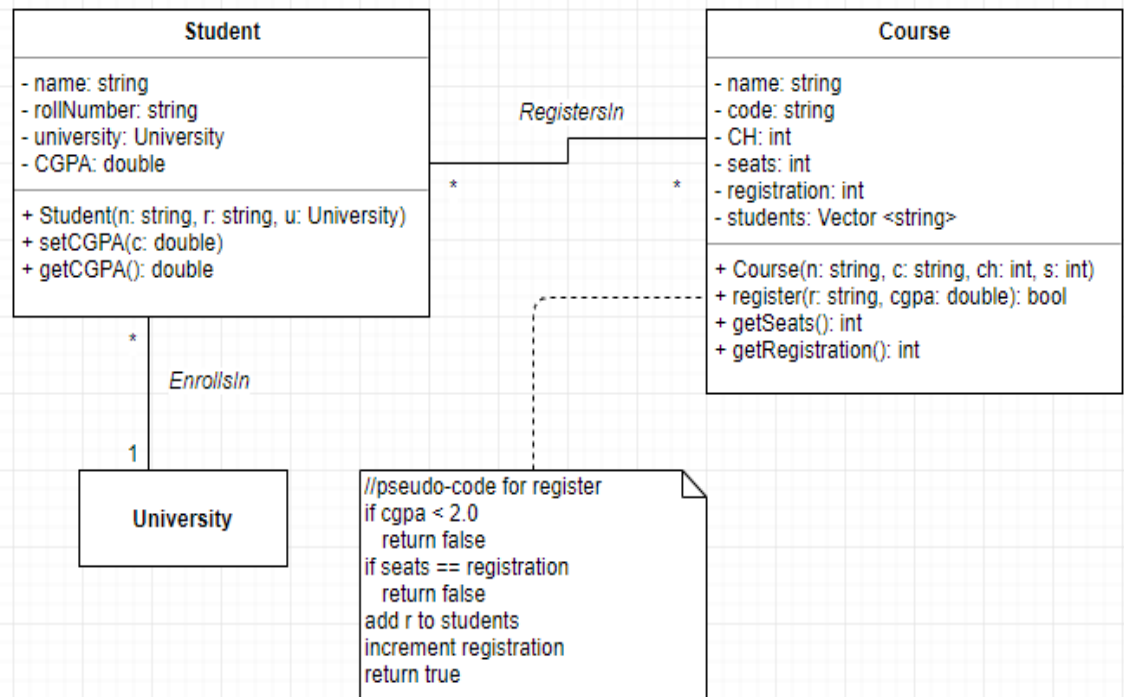
Which component is responsible to set a particular value for the common variable?

- Tracing back to all components that access those data to evaluate the effect of the change



Modularity: Coupling: Control Coupling

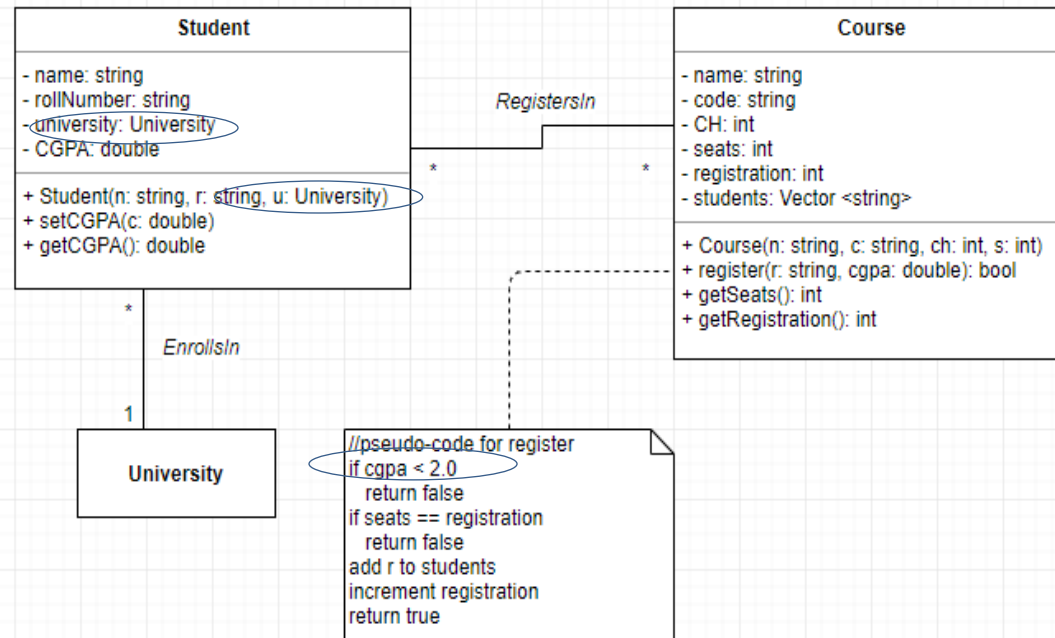
- One component passes parameters to control the activity of another component
- The controller without direct data coupling
- Make each component a module
- Example: Module B, indicating next based on



Modularity: Coupling: Stamp and Data Coupling

- **Stamp coupling** occurs when complex data structures are passed between modules.

- Stamp coupling occurs because the modules share data structures.
- Example: Module A passes a stamp to Module B, which then uses it.



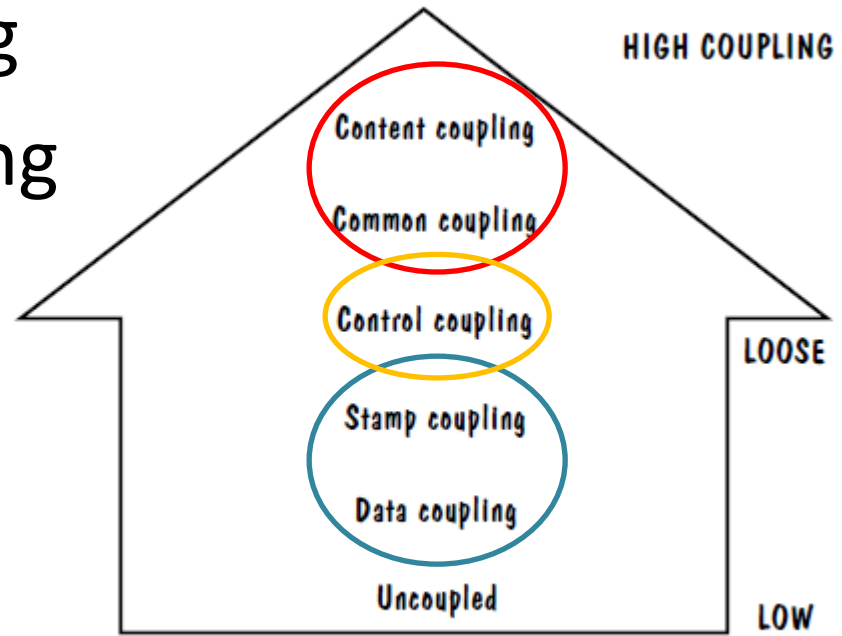
- If only data is passed between the modules:

- Data coupling is represented by a simple line between modules.
- Only data values are passed.
- Easiest to trace.
- Example: Module A passes data directly to Module B without any knowledge of how Module B uses that data internally.

See Pressman for Type Use coupling

Modularity: Coupling: Types of Coupling

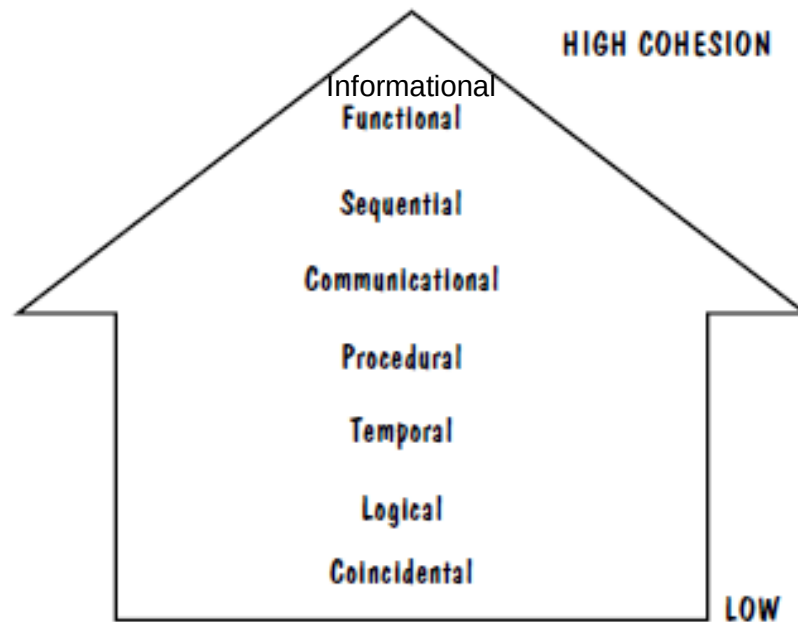
- Content coupling
- Common coupling
- Control coupling
- Stamp coupling
- Data coupling
- Uncoupled



High coupling is **NOT** desired

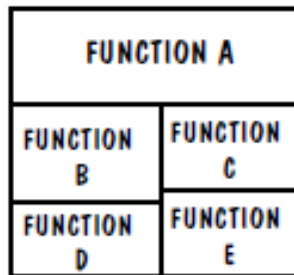
Modularity: Cohesion: Types of Cohesion

- **Cohesion** refers to the dependence within and among a module's internal elements (e.g., data, functions, internal modules)



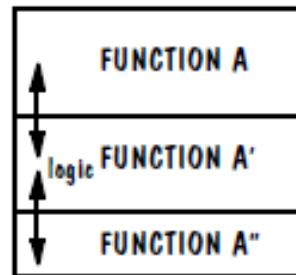
Low cohesion is not desired

Modularity: Cohesion: Types of Cohesion



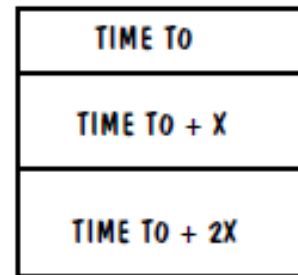
COINCIDENTAL

Parts unrelated



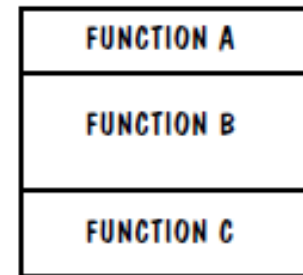
LOGICAL

Similar functions



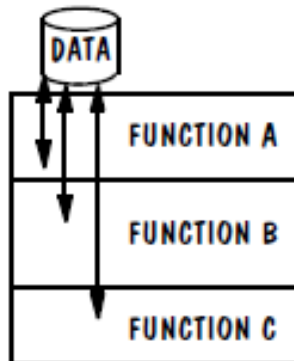
TEMPORAL

Related by time



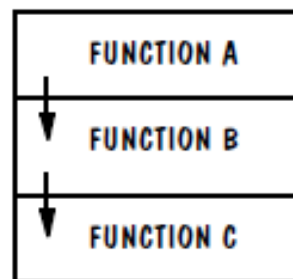
PROCEDURAL

Related by order of functions



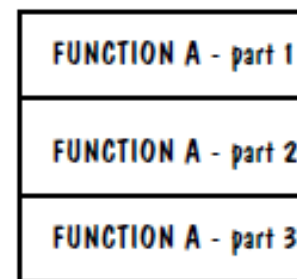
COMMUNICATIONAL

Access same data



SEQUENTIAL

Output of one part is input to next



FUNCTIONAL

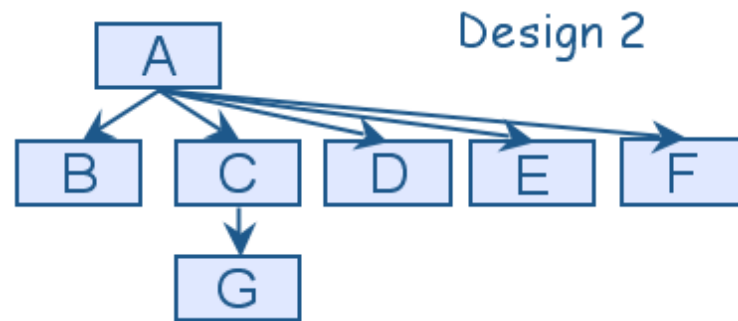
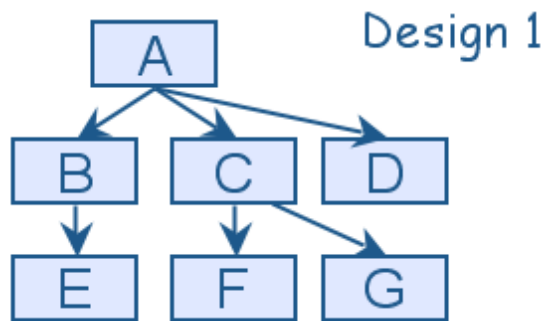
Sequential with complete, related functions

Incremental Development

- Given a design consisting of software units and their interfaces, we can use the information about the units' dependencies to devise an incremental schedule of development
- Start by mapping out the units' **uses relation**
 - relates each software unit to the other software units on which it depends
- **Uses graphs** can help to identify progressively larger subsets of our system that we can implement and test incrementally

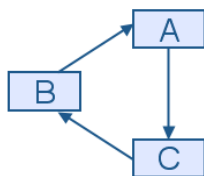
Incremental Development

- Uses graphs for two designs
 - **Fan-in** refers to the number of units that use a particular software unit
 - **Fan-out** refers to the number of units used by particular software unit

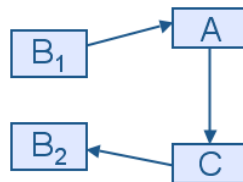


Incremental Development

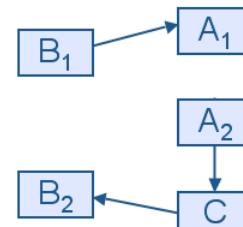
- We can try to break a cycle in the uses graph using a technique called **sandwiching**
 - One of the cycle's units is decomposed into two units, such that one of the new units has no dependencies
 - Sandwiching can be applied more than once, to break either mutual dependencies in tightly coupled units or long dependency chains



-(a)



-(b)



-(c)

Abstraction

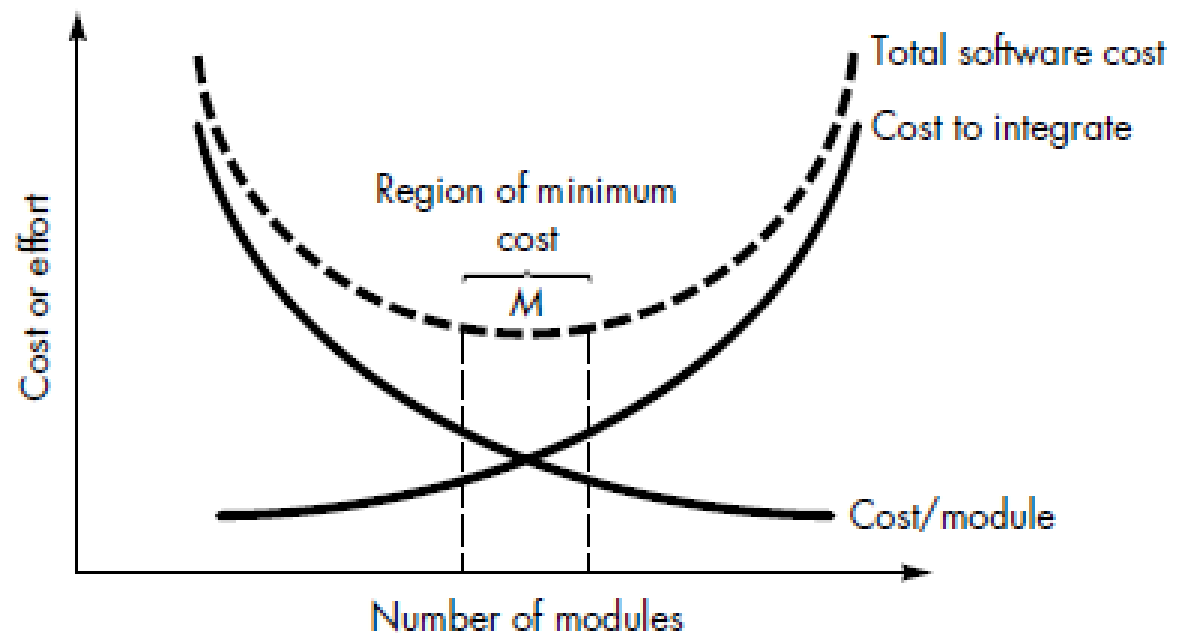
- An **abstraction** is a model or representation that omits some details so that it can focus on other details
- The definition is vague about which details are left out of a model, because different abstractions, built for different purposes, omit different kinds of details in order to focus on one part/aspect and avoid distractions

Generality

- **Generality** is the design principle that makes a software unit as universally applicable as possible, to increase the chance that it will be useful in some future system (reusability)
- Amortize the cost of module development
- We make a unit more general by increasing the number of contexts in which it can be used. There are several ways of doing this:
 - Removing preconditions
 - Simplifying postconditions

Applying Design Principles

- Modularity?
 - over/under modularize?



References

- Pfleeger Book slides from UCF
- Bruegge et al. OOSE book
- Book and slides from Pressman
- <http://www.authorstream.com/Presentation/umr17-1645231-pressman-ch-12-user-interface-design/>

Acknowledgement!

- Some slides have been reused from UCF slides for the SE course, specially for UI design